

**QRCS Corporation**

# **SHAKE Cost Based KDF Protocol Analysis**

**Title:** Implementation Analysis of the SHAKE Cost Based Key Derivation Function (SCB)

**Author:** John G. Underhill

**Institution:** Quantum Resistant Cryptographic Solutions Corporation (QRCS)

**Date:** November 2025

**Document Type:** Technical Cryptanalysis Report

**Revision:** 1.0

## **Chapter 1: Introduction**

### **1.1 Purpose**

This document provides a comprehensive implementation cryptanalysis of the SHAKE Cost Based (SCB) key derivation function as specified in the SCB specification and implemented in the SCB source code. The purpose of this analysis is to examine the correctness, security properties, operational reliability, and compliance of the implementation with the formal design goals outlined in the specification. The assessment includes a detailed review of the construction, the data processing model, memory and computational hardness mechanisms, and all internal and external interfaces provided by the implementation.

### **1.2 Scope of Analysis**

The scope of this analysis covers:

- The SCB specification at version 1.1a
- All implementation files associated with SCB, including scb.c, scb.h, memutils, and the supporting Keccak SHAKE code
- The full operational path of SCB including initialization, iterative memory hard derivation, key updates, and output generation
- Security and correctness characteristics that arise from implementation decisions

Out of scope for this document are formal mathematical proofs and reductions. These appear in the formal cryptanalysis document.

### **1.3 Relation to the SCB Specification and Code**

This analysis directly references the SCB specification, which defines the functional intent and requirements for SCB, and cross compares those requirements with the corresponding implementation in the source code. Every mechanism described in the specification is evaluated against the implementation for consistency, correctness, and completeness. Discrepancies, if present, are identified and analyzed. The goal is to ensure that the implemented behavior matches the intended design and that no unintentional side effects reduce security.

### **1.4 Cryptographic Context and Intended Use Cases**

SCB is designed as a cost based key derivation function suitable for environments that require strong resistance to brute force and parallelized attacks. It is intended for use in password hashing, seed expansion for cryptographic constructions, and any context where attacker cost must be increased through memory or computation. SCB uses SHAKE extendable output functions as its cryptographic core and introduces memory hard and compute hard components that significantly increase attack cost on commodity hardware.

The design assumptions, operational constraints, and cryptographic primitives align SCB with the class of modern memory hard derivation functions that address the limitations of purely computational KDFs. This analysis focuses on the implementation level considerations that determine whether SCB satisfies these goals in practice.

## **2. Architectural Overview of SCB**

### **2.1 Structural Model**

SCB is structured as a sequence of dependent operations that transform a seed into hardened key material through both computational and memory bound processing. The architecture consists of three primary components:

1. A cSHAKE based initializer that absorbs the seed and personalization string.
2. A memory filling system that uses a scatter pattern to enforce memory hardness.

3. An iterative key evolution mechanism that ties each round of work to all previous rounds.

These components function as a continuous pipeline rather than as isolated routines.

## **2.2 Role of the Seed and Initial Sponge State**

The architecture begins with absorption of the seed and optional personalization string. This step defines the initial Keccak state and selects the SHAKE rate used for all subsequent operations. The seed's length determines whether SCB operates in SHAKE 256 or SHAKE 512 mode, and the personalization string establishes domain separation for independent applications.

This produces the initial derived key, which seeds all later processing.

## **2.3 Memory Engine and Scatter Pattern**

The memory engine is the defining element of SCB hardness. It allocates a buffer equal to memcost mebibytes and divides it into 64-byte cache lines. Instead of filling this buffer linearly, SCB computes a deterministic scattering pattern that forces each write to a location separated from the previous write by a distance that exceeds the L2 cache size.

This pattern serves two purposes:

- It ensures consistent cache misses at each memory access.
- It prevents attackers from establishing locality or reordering accesses to benefit from hardware acceleration.

The memory engine is designed to be expensive on any architecture, including GPUs and ASICs.

## **2.4 Hash State Coupling and Iterative Evolution**

Alongside memory filling, SCB maintains a second Keccak state that absorbs iteration counters, memory indices, and periodic full buffer contents. This creates a dependency graph in which:

- Each iteration depends on the buffer contents.
- The buffer contents depend on SHAKE output.
- SHAKE output depends on the derived key.

- The derived key depends on the previous iteration.

The result is a strictly serial structure with no exploitable shortcut.

## **2.5 Output Extraction and Finalization**

After completing the required number of iterations, SCB finalizes the accumulated hash state to produce a new derived key. This key seeds a SHAKE instance that generates arbitrarily long output material. The entire sequence is deterministic, and its runtime is proportional to both cpucost and memcost.

## **3. Inputs, Parameters, and Operational Context**

### **3.1 Seed Requirements**

SCB begins with a caller supplied seed that determines the security level and the internal rate of the Keccak sponge. The implementation requires the seed to be either 32 bytes or 64 bytes in length. A 32-byte seed selects the SHAKE 256 rate, and a 64-byte seed selects the SHAKE 512 rate. This choice affects the total number of Keccak permutations required during memory filling and buffer hashing. The seed should be uniformly random when SCB is used as a general purpose KDF, and it should be the output of a secure password preprocessing step when the caller supplies lower entropy material.

The seed forms the primary cryptographic input to SCB. It is absorbed into the Keccak state during initialization, and it determines the initial derived key that seeds the memory engine. All later stages depend on this value, so the entropy quality of the seed directly influences the hardness of the final output.

### **3.2 Personalization String and Domain Separation**

SCB supports an optional personalization string that allows callers to domain separate independent uses of the KDF. The personalization string is absorbed immediately after the seed during initialization. Its purpose is to guarantee that two applications using identical seeds but distinct personalization values will derive unrelated keys. This behavior mirrors the customization features found in cSHAKE and is essential for multi-tenant or multi-protocol environments.

The personalization string does not alter the selection of SHAKE variant or cost parameters, but it contributes to the uniqueness of the derived key and should be selected with awareness of application-level context.

### **3.3 Computational Cost Parameter**

The cpucost parameter controls how many full iterations of the SCB pipeline must be completed before output is produced. Each iteration consists of initializing a working SHAKE instance with the current derived key, filling the memory buffer according to the scatter pattern, absorbing counters and memory indices, performing periodic full buffer absorption, and finalizing the state to produce a new key.

Because each iteration depends on the derived key produced by the one before it, cpucost introduces a strictly serial dependency chain. Increasing cpucost increases the computational burden in a predictable linear manner. Applications should select cpucost according to their performance constraints and threat model, balancing latency against desired attacker cost.

### **3.4 Memory Cost Parameter**

The memcost parameter controls the size of the memory buffer that SCB allocates during the derivation process. The buffer contains memcost mebibytes, divided into 64-byte cache lines. This buffer dominates the hardness properties of the system. Because SCB enforces a scattered access pattern designed to exceed L2 cache capacity, large memcost values require constant access to high latency memory locations.

Memory cost affects both performance and security. Larger memcost values increase the difficulty of parallel attack strategies, impose substantial bandwidth requirements, and limit the attacker's ability to run many derivations concurrently. The value must be chosen with awareness of the system's available memory and the acceptable runtime for legitimate use.

### **3.5 Output Requirements**

After completing all iterations, SCB produces output through a SHAKE based expand phase. Output length is determined entirely by the caller, and there is no defined maximum length beyond practical memory constraints. Because SHAKE is an extendable output function, SCB is capable of generating arbitrarily long pseudo random sequences from the final derived key.

The implementation requires the caller to supply a buffer of the desired length. SCB fills this buffer by squeezing the internal Keccak state until the requested number of bytes is produced.

### **3.6 Operational Constraints and Intended Use**

SCB is designed for environments that require significant resistance against brute force search, large scale parallel attacks, or hardware acceleration. It is suitable for password hardening, key stretching, seed transformation, and general purpose KDF applications where deterministic output is required and substantial cost must be imposed on the adversary.

Correct operation depends on selecting appropriate values for cpucost and memcost, providing seed material of acceptable entropy, and managing personalization strings consistently across systems that rely on domain separation. The effectiveness of SCB relies on these operational decisions as much as on the internal cryptographic mechanics.

## **4. Internal State and Component Interaction**

### **4.1 Overview of the SCB Context Structure**

The behavior of SCB is defined by a compact but carefully organized internal context. This context contains the Keccak sponge state used for initialization, a derived key that evolves across iterations, and the values that determine computational and memory hardness. Each element of the context contributes to one of the system's three major phases: initialization, cost-based transformation, and final expansion.

The context is initialized before any generation operations occur and is disposed after use. It is not intended to be modified or reused outside of these defined phases. SCB relies on the deterministic and tightly controlled evolution of this internal structure to ensure correctness and security.

### **4.2 The Initialization Sponge State**

Initialization begins with a Keccak state prepared in cSHAKE mode. The caller supplied seed is absorbed first, followed by the optional personalization string. This establishes the foundation for all subsequent behavior. The internal Keccak state absorbs these

values according to the cSHAKE construction defined in standard references, providing domain separation and ensuring that no two callers using distinct seeds or personalization strings can produce identical internal states.

After absorption, the sponge is squeezed to produce the initial derived key. This initial key is not a final output; it serves instead as the seed for the working SHAKE instance that drives memory filling. Because the working SHAKE state is recreated for each iteration, this initial key defines the full state trajectory for the remainder of the derivation cycle.

### **4.3 The Derived Key and Its Evolution**

The derived key, stored in the context as `ckey`, is the internal value that transitions from iteration to iteration. It is first produced during initialization, then replaced at the end of each `cpucost` iteration. This evolution is central to the serial nature of SCB. Each update to `ckey` depends on the memory buffer contents and on the counters and indices absorbed into the hash state during the iteration.

This relationship guarantees that the key at iteration  $n$  cannot be computed without performing all the steps of iteration  $n$  minus one. It also ensures that the complete internal state of the algorithm is cryptographically linked to both memory and computation. The derived key therefore acts as both the target of the work performed during iteration and the starting point for the next round of memory filling.

### **4.4 Working SHAKE Instance and Memory Filling Engine**

During each `cpucost` cycle, the implementation initializes a temporary SHAKE instance seeded with the current derived key. This working instance is entirely separate from the initialization sponge or the final expansion sponge. Its purpose is to generate 64-byte blocks that populate the memory buffer.

The buffer is divided into cache line sized segments, and the working SHAKE instance produces one such segment at a time. Each segment is written to a location determined by the scatter index that SCB computes based on the buffer size and the target L2 cache parameter. Because this SHAKE instance is reinitialized each iteration, its output is tightly controlled by the derived key. The memory engine, in turn, controls how this output interacts with system memory.

The working SHAKE instance is temporary, deterministic, and isolated from the rest of the SCB state, which helps prevent unintended interactions or reuse errors.

#### **4.5 Hash Accumulation State**

Alongside the memory filling engine, SCB maintains a second Keccak state referred to in the implementation as the hash state. While the working SHAKE instance produces content for the buffer, the hash state absorbs metadata about the process. It takes in: the index of each memory write, the iteration counter, the entire buffer at periodic intervals.

This absorption strategy ensures that every part of the memory filling process contributes to the final derived key. The hash state accumulates these inputs and is finalized at the end of each iteration. The result replaces the current ckey, completing the link between memory operations and state evolution.

#### **4.6 Interaction of Components Across Iterations**

The components described above interact in a predictable and tightly controlled manner. Initialization defines the starting state. The derived key produced in that phase seeds the working SHAKE instance. The working SHAKE instance produces blocks that fill the memory buffer. The hash state absorbs information about these operations. The finalized hash state then produces a new derived key, which feeds into the next iteration.

This arrangement produces a chain of dependencies:

1. Seed and personalization define the initialization state.
2. The initialization state defines the first derived key.
3. The derived key defines the working SHAKE output.
4. The working SHAKE output defines the memory buffer contents.
5. The memory buffer defines the hash state inputs.
6. The hash state defines the next derived key.

These interactions form a linear sequence with no branches or shortcuts. The architecture guarantees that an attacker must reproduce every step of the process to compute the final output.



## 4.7 Final Expansion State

When all iterations have been completed, the final derived key becomes the seed for the last Keccak operation. This operation is a standard SHAKE expansion, and it produces the caller specified amount of output. The derived key influences this output directly, and the output is deterministic given all previous operations.

Because the expansion occurs only after the full SCB cycle is completed, the final SHAKE operation must always be preceded by the full cost-based process.

## 5. Memory Hardness Mechanism

### 5.1 Overview

The most distinctive feature of SCB is its intentional reliance on memory hardness as a primary security property. The design forces the algorithm to interact with memory in a way that isolates each write to a region that cannot be buffered in low latency cache. This significantly increases the cost of evaluating the function and prevents attackers from relying on parallel hardware to accelerate computations. The memory hardness mechanism in SCB is implemented through a large working buffer and a scattering pattern that ensures nonlocal access across the full buffer range.

### 5.2 Buffer Allocation and Layout

SCB allocates a working buffer whose size is determined entirely by memcost. The buffer occupies memcost mebibytes and is divided into 64-byte segments. These segments correspond to the cache line size assumed by the implementation. Although the size of a cache line can vary across architectures, the reference values used in SCB are broad enough to capture common hardware patterns while still ensuring that memory traffic remains unpredictable at the cache level.

The layout of the buffer is simple. It is a contiguous region divided into fixed size segments, and the algorithm assigns a logical index to each segment. These indices allow the scattering mechanism to determine the order in which the segments are filled.

### 5.3 Scatter Index Generation

The implementation computes a scatter index sequence that dictates the order in which SCB will write to the memory buffer. This sequence is not random; it is entirely

deterministic given memcost. The purpose of the sequence is to ensure that consecutive writes occur at offsets separated by distances that exceed the capacity of the L2 cache.

The scattering logic examines the size of the buffer, the number of cache lines it contains, and the target L2 cache size. It then computes a lane based interleaving pattern that spreads logical access across the full memory range. This pattern is designed so that if a write occurs at a location near the beginning of the buffer, the next logical write will occur at a location well beyond the range that the L2 cache can retain. This forces the CPU to issue a new memory request for each access.

As a result, the buffer cannot be effectively prefetched or cached, and an attacker cannot rearrange writes in a more favorable order.

#### **5.4 Memory Filling Process**

During each cpucost iteration, SCB initializes a working SHAKE instance seeded with the current derived key. This instance produces a sequence of 64-byte blocks. Each block is written to the memory buffer at the offset specified by the scatter index. The algorithm proceeds from the first logical index to the last, following the scattering pattern exactly.

This procedure ensures three important properties:

1. Each write incurs a cache miss.
2. Each write depends on the output of the working SHAKE instance, which depends on the derived key.
3. The full memory buffer must be processed to complete an iteration.

Because the buffer is large and the scattering pattern is wide, every access forces the processor to fetch data from memory instead of benefiting from cache locality.

#### **5.5 Periodic Full Buffer Absorption**

After writing a fixed number of cache lines, SCB absorbs the entire buffer into the hash state. This operation binds the buffer contents directly into the cryptographic state driving the evolution of the derived key. The interval at which these full buffer absorptions occur corresponds to the number of cache lines that fit into the L2 cache. The intent is to prevent adversaries from storing compressed or partial representations of the buffer, since the buffer must be fully processed regardless of its contents.

Absorbing the entire buffer at predictable intervals ensures that an attacker who attempts to reduce memory usage would lose access to the necessary material required to produce the next derived key. This requirement eliminates most time memory tradeoff strategies.

## **5.6 Dependence on System Memory Architecture**

The design of the memory hardness mechanism assumes a system with distinct cache layers. SCB distributes memory operations across ranges that exceed these layers to enforce high latency. While the specific sizes of L1, L2, and L3 caches vary among processors, the SCB scattering pattern uses values that reflect common architectural constraints. This makes SCB broadly effective across modern platforms.

The design performs well even when memory architectures differ. Systems with large caches do not benefit significantly because the scattering pattern ensures that access distances exceed even generous L2 cache sizes. Systems with smaller caches experience even stronger enforcement of memory hardness.

## **5.7 Memory Hardness as a Security Property**

By combining a large buffer, scattered access, and buffer wide absorption, SCB imposes a memory cost that cannot easily be reduced or bypassed. The algorithm prevents attackers from:

- compressing memory and recomputing only small regions
- reorganizing writes into a more cache friendly order
- computing segments of the function in parallel
- performing speculative evaluation without full memory state

The memory hardness mechanism reinforces the serial nature of SCB and ensures that every evaluation incurs a substantial and unavoidable memory penalty.

# **6. Computational Hardness and Sequential Dependency**

## **6.1 Overview**

Computational hardness in SCB arises from the substantial amount of Keccak based processing required to generate and evolve the internal state, combined with strict ordering constraints that eliminate opportunities for parallel execution. Although memory hardness is the dominant cost factor, SCB also ensures that every iteration requires a substantial number of Keccak permutations and state transitions. These operations contribute directly to the overall hardness of the function and provide resistance against attackers with high-speed hardware.

## **6.2 The Role of Keccak Permutations**

During each cpucost iteration, SCB performs a large number of Keccak permutations. These permutations arise in two places. The working SHAKE instance repeatedly generates 64-byte blocks used to populate the memory buffer, and the hash state continuously absorbs metadata about the filling operation. Both actions require full permutations at the rate defined by the selected SHAKE variant.

Because the working SHAKE instance is seeded with the current derived key, each block produced depends on the results of the prior iteration. This makes the sequence of permutations strictly deterministic and dependent on the full history of the derivation process. The number of permutations grows proportionally with the size of the memory buffer and the value of cpucost, and cannot be reduced without invalidating the final derived key.

## **6.3 Sequential Data Flow**

SCB enforces a strict ordering on its operations. Each iteration requires information produced by the previous iteration. Each memory write requires output produced by the working SHAKE instance. Each absorption step requires the memory indices and counters produced at that point in the process. There is no place in the design where SCB introduces parallel branches or paths that can be evaluated independently.

This serial structure ensures that performance scales linearly with cpucost. It also ensures that no hardware accelerator can evaluate multiple iterations in parallel. Even when attackers possess the ability to run many Keccak permutations concurrently, SCB prevents them from exploiting this ability, because the input to each permutation is tied to the results of the previous permutation.

## **6.4 Interdependence of Memory and Computation**

Although memory hardness defines the majority of SCB's resistance to attack, the computational aspects reinforce that hardness by binding memory writes to the SHAKE output and linking those writes into the derived key. The connection between memory activity and computational processing is essential. An attacker seeking to reduce memory costs must still perform the computational steps, and an attacker seeking to reduce computational cost must still perform the memory steps. Neither component can be short circuited.

This interdependence also creates a situation in which the working SHAKE state cannot be optimized or predicted in advance. Because ckey changes with each iteration, the sequence of SHAKE outputs changes as well. Memory writes cannot be predicted or precomputed without knowledge of the new derived key, and the new derived key cannot be computed without first completing the memory operations. As a result, computation and memory are inseparable parts of the same structure.

### **6.5 Impact of cpucost on Hardness**

The cpucost parameter directly affects the amount of sequential work SCB requires. Larger values of cpucost increase the number of times the algorithm must:

1. Initialize the working SHAKE instance.
2. Populate the memory buffer according to the scattering pattern.
3. Absorb memory indices and counters into the hash state.
4. Finalize the hash state to produce a new derived key.

Because each iteration depends fully on the derived key from the previous iteration, cpucost creates a chain of operations whose length defines the overall computational effort. The attacker must evaluate the entire chain to reach the correct final state. There is no point where partial progress can be cached or reused across derivations with different seeds or personalization strings.

### **6.6 Infeasibility of Parallel Evaluation**

SCB is resistant to parallel processing not only because of memory hardness, but because of its computational structure. Attacks relying on GPUs or custom hardware are limited by two constraints:

- the scattering pattern prevents parallel gathering of memory regions, and

- the working SHAKE instance prevents generation of widely separated memory blocks in parallel.

Even if an attacker could evaluate Keccak permutations at very high speeds, SCB forces those permutations to be applied in a predetermined order. No permutation can be skipped or computed out of sequence without breaking the chain of dependencies that define the evolution of the derived key.

## **6.7 Contribution to Overall Security**

Computational hardness ensures that SCB remains expensive for attackers regardless of their memory resources. Although modern systems with high memory bandwidth may be able to manage large memcost values, the additional computational load from repeated Keccak permutations provides an independent layer of protection. Together, the computational and memory hardness components guarantee that SCB maintains a consistent and reliable cost profile across diverse hardware environments.

# **7. Detailed Operational Flow**

## **7.1 Overview**

The SCB derivation process unfolds as a sequence of tightly connected stages. Each stage produces state material that is required by the next, which enforces the strict serial nature of the construction. The operational flow described here reflects the actual behavior of the reference implementation and remains consistent with the structure defined in the SCB specification. Understanding this flow is critical for evaluating correctness, performance, and security properties.

## **7.2 Initialization Phase**

The derivation sequence begins with initialization of the SCB context. The caller provides a seed of fixed length and an optional personalization string. These values are absorbed into a Keccak sponge using the cSHAKE customization method. The seed length determines the internal rate of the sponge. The personalization value, when present, provides domain separation.

After absorption, the sponge is squeezed once to produce the initial derived key. This key does not represent final output material but instead serves as the starting point for

the cost-based transformation process. The context records cpucost and memcost and resets any residual state from previous operations to prevent contamination of the new derivation sequence.

### **7.3 Preparation for Iterative Processing**

Before the first iteration begins, SCB prepares two internal structures. The first is the working SHAKE instance, which is seeded with the current derived key and is responsible for generating the 64-byte segments used during memory filling. The second is the iterative hash state, which absorbs metadata about the filling process and drives the evolution of the next derived key.

The working memory buffer is then allocated. The size of this buffer is determined entirely by memcost and is divided into cache line sized segments. The scatter index array is computed immediately afterward. This array determines the order in which segments will be written during the iteration.

### **7.4 Memory Filling Stage**

The memory filling stage represents the core of the SCB workflow. The working SHAKE instance produces one block at a time, each block being exactly the size of a cache line. For each logical index, SCB retrieves the corresponding physical offset from the scatter index array. The block is written to the buffer at that location, and the iterative hash state absorbs data that encodes both the index and the iteration count.

This procedure continues until every cache line has been written. Because the scatter pattern is constructed to exceed cache boundaries, each write operation forces an access to higher latency memory. The process maintains a stable and predictable order. No shortcuts are available, and each block must be generated and placed in the correct location before proceeding to the next.

### **7.5 Periodic Full Buffer Absorption**

At intervals that correspond to the size of the L2 cache, SCB absorbs the entire memory buffer into the iterative hash state. This step binds the buffer contents to the derived key evolution process. The reason for this periodic absorption is to ensure that an attacker cannot compute the derived key without reconstructing the full buffer. The entire memory must be retained and processed for each iteration.

This absorption step persists across all iterations. It forms one of the mechanisms by which SCB enforces memory hardness across the entire derivation sequence.

## **7.6 Iteration Finalization**

Once the memory buffer has been fully written and all required buffer absorptions have been completed, SCB finalizes the iterative hash state. The result of this finalization is a new derived key, which replaces the previous internal key. This stage marks the boundary of a single cpucost iteration.

If cpucost is greater than one, SCB reinitializes the working SHAKE instance using the newly produced derived key and proceeds to the next iteration. The sequence is repeated for as many iterations as specified by the caller.

## **7.7 Post Processing and Output Generation**

After all iterations have completed, the final derived key forms the basis for the expansion phase. SCB initializes a final SHAKE instance with this key and squeezes output until the caller specified number of bytes has been produced. The expansion stage does not involve memory filling or scattering. It exists purely to convert the final key into output material of arbitrary length.

This final stage completes the derivation. The context may then be disposed, which clears the internal state and prevents reuse.

## **7.8 Deterministic Flow and Reproducibility**

The operational flow described above is entirely deterministic. Given the same seed, personalization string, cpucost, memcost, and output length, SCB will produce the same result on every execution. The strict sequencing of memory writes, sponge absorption, hash finalization, and expansion ensures reproducibility. This property is essential for systems that rely on deterministic key generation, verification, or synchronization.

## **7.9 Significance of Sequential Dependency**

The entire operational flow is built around the requirement that each stage depends completely on the results of the previous stage. The working SHAKE instance cannot be reinitialized until the previous derived key has been produced. The derived key cannot be produced until the iterative hash state has absorbed all required memory-based metadata. The iterative hash state cannot be constructed without the memory buffer



and scatter pattern. The memory buffer cannot be filled without the working SHAKE output.

This structure forces SCB to operate as a single continuous chain of dependent computations. It prevents parallel shortcuts and ensures that the algorithm's cost profile remains consistent and resistant to hardware acceleration.

## **8. Code Level Alignment and Verification**

### **8.1 Overview**

The SCB implementation follows a compact and modular design that reflects the structure defined in the specification. Each component of the reference code corresponds to a defined stage in the SCB workflow. The purpose of this chapter is to verify that the behaviors described earlier are faithfully implemented and that no unintended shortcuts, inconsistencies, or side effects appear in the code. This verification also confirms that the implementation enforces the deterministic, sequential, and cost based nature of the SCB construction.

### **8.2 Initialization Logic**

The initialization function validates the seed length and ensures that cpucost and memcost fall within the defined bounds. Once parameters are validated, the function selects the correct SHAKE rate based on seed length. It then clears any residual context state and performs the cSHAKE style absorption of the seed and personalization string. The code matches the specification in both order and method.

After absorption, the implementation squeezes the sponge once to produce the initial derived key. This output is stored in the context and becomes the seed for later memory filling operations. The code does not expose this value externally and clears temporary buffers after use, which is consistent with secure handling practices.

### **8.3 Working SHAKE Instance Construction**

During each cpucost iteration, the implementation constructs a new working SHAKE instance seeded with the current derived key. This instance produces fixed size blocks that populate the memory buffer. The code correctly initializes and disposes this temporary SHAKE object and does not permit reuse across iterations. This behavior

aligns with the specification's requirement that each iteration must be independent, except through the derived key.

The internal rate and block size are consistent with the selected SHAKE variant. No divergence from the expected Keccak usage patterns is present.

#### **8.4 Scatter Index Generation**

The scatter index generator computes a deterministic mapping from logical indices to physical buffer offsets. The implementation calculates the number of lanes and the number of cache lines per lane in the same manner specified in the SCB design notes. The resulting pattern ensures that consecutive writes occur at offsets that exceed the assumed L2 cache size.

The code uses integer arithmetic that is consistent with the design assumptions. The function allocates, populates, and clears its index array in a well-structured and safe manner. The generated index pattern matches the intended behavior exactly.

#### **8.5 Memory Filling Mechanism**

The memory filling function retrieves the scatter index array and uses the working SHAKE instance to generate one cache line sized block at a time. Each block is written to the location designated by the corresponding scatter index entry. After each write, the code updates the iterative hash state with encoded index information. This coupling between memory writes and the hash state is central to the design and is implemented correctly.

The memory fill loop processes each logical index in sequence, ensuring that no write is skipped or reordered. The code does not contain alternative write paths or shortcuts that could reduce memory activity. Buffer writes are performed safely, with bounds respected at every step.

#### **8.6 Full Buffer Absorption**

After processing a number of cache lines equal to the assumed L2 cache capacity, the code absorbs the entire buffer into the iterative hash state. This behavior appears in the implementation exactly as described in the specification. The absorption call processes the full buffer in its current state, and the procedure is repeated at every required interval.

The implementation correctly uses the hash update interface and clears temporary buffers after each update. This demonstrates careful attention to both correctness and secure memory handling.

### **8.7 Iteration Finalization and Key Evolution**

At the end of each cpucost iteration, the implementation finalizes the iterative hash state. The result is written into the context as the new derived key. This behavior mirrors the specified transition logic, where each iteration relies on the derived key produced by the one before it.

The code disposes the hash state after finalization and clears intermediate data structures. This eliminates the possibility of key material leaking through unintended state persistence.

### **8.8 Output Generation**

After completing all required iterations, the final derived key is passed to a SHAKE instance used solely for output generation. The implementation squeezes this state until the caller supplied output buffer is filled. This final SHAKE usage is cleanly separated from earlier phases and does not retain internal memory buffers.

The code handles output generation consistently and does not impose unnecessary constraints on output length. This behavior is correct with respect to the specification, which describes SCB as an extendable output KDF.

### **8.9 Memory Handling and Sanitization**

The reference implementation takes appropriate measures to prevent sensitive information from remaining in memory after it is no longer needed. Buffer contents, index arrays, and temporary states are cleared using the memory utility functions. The context disposer reliably resets ckey, cost parameters, and all Keccak states.

These practices match the expectations for secure cryptographic code and align with the specification's emphasis on controlled lifecycle behavior.

### **8.10 Alignment Conclusion**

The implementation matches the SCB specification across all functional layers. Each stage of the derivation process is present, ordered correctly, and uses Keccak based primitives in accordance with their documented behavior. The memory scattering

mechanism is implemented exactly as intended, and the serialization of iterations matches the sequential dependency described earlier.

The code exhibits no departures from the defined design and does not introduce weaknesses through incorrect ordering, skipped steps, or inaccurate parameter handling. The implementation therefore meets the expected standard of correctness for a secure cost based key derivation function.

## **9. Correctness and Consistency Analysis**

### **9.1 Overview**

The correctness of the SCB implementation depends on coherent interaction among its sponge-based components, its memory heavy transformation process, and its key evolution mechanism. This chapter examines whether the implementation behaves exactly as intended, whether each transformation step is complete, and whether the design avoids inconsistencies that could weaken security or produce unintended outputs. By analyzing the internal logic of the implementation, we confirm that SCB operates deterministically and remains faithful to the model described in the specification.

### **9.2 Correctness of Input Handling**

The implementation validates seed length and cost parameters before proceeding with any derivation work. Seed length is checked against the permitted values of 32 and 64 bytes. cpucost and memcost are verified to fall within their acceptable ranges. These checks enforce the preconditions described in the specification and prevent undefined behavior. Inputs that fail validation are rejected early, ensuring that the internal state never enters a partially initialized or inconsistent configuration.

The personalization string is accepted as optional and absorbed directly into the initialization sponge after the seed. This behavior matches the specification's requirement for domain separated customization.

### **9.3 Correctness of State Initialization**

The initialization logic correctly clears the SCB context prior to absorbing new inputs. This ensures that residual state from a previous derivation cannot influence a new sequence. The cSHAKE style absorption of the seed and personalization string is

implemented in the correct order, and the initial derived key is produced by squeezing the initialized sponge exactly once. Temporary buffers associated with this step are cleared upon exit, which prevents unintended exposure of intermediate values.

The context records the selected SHAKE rate, the seed length, and the cost parameters. These values determine the behavior of later stages and are stored consistently.

#### **9.4 Correctness of the Memory Filling Process**

The memory filling mechanism relies on the working SHAKE instance and the scatter index array to generate and place content across the entire buffer. The implementation processes indices in deterministic order, generates one cache line sized block at a time, and writes each block to the physical location defined by the scatter index. The loop iterates across the full number of cache lines, ensuring that no region of the buffer is left uninitialized.

The iterative hash state absorbs each write's metadata, including the logical index and the iteration counter. These updates follow the specification requirement that all memory activity must influence the next derived key. The code contains no alternative write paths and includes no conditional logic that would skip indices. As a result, the buffer is filled consistently and completely.

#### **9.5 Correctness of Full Buffer Absorption**

The periodic absorption of the entire memory buffer into the iterative hash state is a critical part of the SCB design. The implementation performs this operation exactly at the intervals specified by the assumed L2 cache capacity. During absorption, every byte of the buffer is processed, and the absorption path follows the Keccak update logic as intended.

This step eliminates any possibility of an attacker shortcutting memory by keeping only partial representations of the buffer. Because the full buffer is required for hash finalization, correctness in this part of the code is essential. The implementation satisfies this requirement and clears temporary data used during absorption.

#### **9.6 Correctness of Iteration Finalization**

At the end of each cpucost iteration, the implementation correctly finalizes the iterative hash state and writes the resulting output into the derived key buffer. This output replaces the previous key without delay or reuse. The finalization step follows the

expected pattern of absorbing all pending data and applying the permutation required to produce a new state.

The code clears the hash state after finalization, which prevents cross iteration contamination or accidental key reuse. This matches the intended model for producing a serially dependent chain of derived keys.

## **9.7 Correctness of Output Generation**

Once the final derived key is produced, SCB initializes a final SHAKE instance for output expansion. The output generation logic is consistent with the behavior described in the specification. It uses the derived key as input and squeezes blocks of output until the caller's requested length is satisfied. The implementation handles arbitrary output lengths correctly and clears temporary data at the end of the process.

There is no path in the code that permits output generation before completing all cpucost iterations, which enforces the intended cost profile.

## **9.8 Deterministic Behavior and State Consistency**

SCB is designed to be deterministic. The same inputs must produce the same outputs across all platforms that follow the specification. The implementation maintains this property by:

- performing all operations in a fixed order
- eliminating branches dependent on secret data
- avoiding uninitialized memory reads
- maintaining a stable sequence of sponge calls and memory writes

No randomness is introduced at any stage of the workflow, and the shape of the scatter pattern is entirely defined by memcost. Consistency across iterations is preserved through careful memory handling and state updates.

## **9.9 Boundary Condition Behavior**

Boundary conditions are handled correctly across the implementation. Very small memcost values still produce a valid buffer, and cpucost values as low as one still trigger a full derivation cycle. Larger values within allowed limits do not cause integer overflow or buffer overruns. The code uses fixed size integers and precise bounds checks for

buffer lengths and index ranges, which prevent anomalies at edges of valid parameter sets.

The implementation also behaves correctly when the personalization string is empty or of arbitrary length. No inconsistencies arise from input size variation.

### **9.10 Consistency Conclusion**

The SCB implementation matches the intended design and maintains consistency across all internal operations. All input validation, memory handling, state updates, hash absorptions, and finalization steps are complete and correctly structured. There is no evidence of shortcuts, skipped operations, or uninitialized paths that would undermine correctness.

The implementation therefore satisfies the correctness requirements of the SCB specification and demonstrates consistent behavior across all permitted parameter ranges.

## **10. Security Analysis**

### **10.1 Overview**

The security of SCB arises from the combined effect of its computational hardness, memory hardness, and deterministic state evolution. The design forces an adversary to perform the same sequence of memory bound and computation heavy operations required of a legitimate user. SCB resists brute force search, parallel hardware acceleration, time memory tradeoff attacks, and precomputation. This chapter evaluates these properties in detail and examines whether the implementation reliably upholds the intended security guarantees.

### **10.2 Memory Hardness as a Primary Defense**

SCB's most important security property is its resistance to parallel and hardware accelerated attacks through memory hardness. This resistance comes from the requirement that the memory buffer, sized according to memcost, be fully constructed and processed during each cpucost iteration. The scattering pattern forces memory writes to occur at offsets spaced beyond cache capacity. These accesses force high latency operations that cannot be reduced by optimized caching strategies.

Because the order of memory operations is fixed and the locations are derived deterministically from memcost, an attacker cannot rearrange work to reduce latency. No benefit can be gained from cache locality, and the memory bandwidth of the system becomes the limiting factor. This forces attackers with GPU or ASIC hardware to face similar latency constraints, even when the underlying arithmetic capabilities of those devices are significantly greater than those of general-purpose CPUs.

### **10.3 Computational Hardness and Dependency Structure**

Memory hardness works in tandem with computational hardness. The working SHAKE instance must produce a large number of blocks during each iteration, and each block requires Keccak permutations. These permutations cannot be parallelized because the output of each permutation depends on the state produced by all previous absorptions and squeezes.

The chain of dependencies between the memory buffer, the iterative hash state, and the evolving derived key ensures that no computation can occur out of sequence. The working SHAKE instance for iteration  $n$  requires the derived key from iteration  $n$  minus one, which in turn depends on completing all of the memory and absorption operations for that iteration. This serial structure enforces a minimum computational workload and prevents speculative evaluation.

### **10.4 Resistance to Brute Force Attacks**

Brute force attacks require evaluating SCB repeatedly for different candidate seeds or passwords. SCB increases the cost of each evaluation substantially through both its memory and computational requirements. The memory buffer must be allocated, filled, and repeatedly processed. The iterative hash state must absorb information about these operations and produce a new derived key.

An attacker attempting to evaluate SCB at large scale must supply comparable memory resources for every guess. The cost of allocating large buffer regions and performing repeated memory misses places a substantial burden on parallel attackers. For attackers using commodity hardware, the memory and time requirements limit the number of guesses per second. For attackers using specialized hardware, SCB disrupts many GPU and ASIC strategies by making memory the dominant resource rather than arithmetic throughput.

### **10.5 Resistance to Time Memory Tradeoff Attacks**



Time memory tradeoff attacks attempt to reduce memory usage at the cost of increased computational work. SCB prevents these attacks by tying the memory buffer directly into the evolution of the derived key. The algorithm absorbs the entire buffer into the iterative hash state at intervals corresponding to the L2 cache capacity. This means that without the full buffer in memory, an attacker cannot construct the correct input for the final hash state and therefore cannot compute the next derived key.

Because buffer contents depend on the working SHAKE instance and the scatter pattern, any attempt to reduce memory usage forces the attacker to recompute large sections of the buffer. These recomputations would themselves require repeated SHAKE generation and full state evolution. As a result, time memory tradeoff strategies do not reduce total work in a meaningful way.

## **10.6 Resistance to Parallelized Hardware Attacks**

Modern hardware attackers rely heavily on GPUs and dedicated logic engines to evaluate hash functions in parallel. SCB counters these approaches by:

- relying on memory latency rather than arithmetic operations
- enforcing nonlocal memory access
- requiring serialized SHAKE output
- binding memory writes to hash state updates

GPUs benefit from high arithmetic throughput but often have poor performance when required to operate on nonlocal memory with irregular access patterns. The scatter pattern in SCB destroys opportunities for coalesced memory access, which is critical for efficient GPU evaluation. ASICs and FPGAs face similar constraints because implementing large, irregular memory banks in hardware is expensive and often negates the benefits of custom arithmetic logic.

The serial dependency between iterations further limits any attempt to split SCB across multiple processing lanes or hardware units. Even highly parallel systems must perform the operations in order.

## **10.7 Resistance to Precomputation and Dictionary Attacks**

Precomputation attacks depend on constructing large tables of possible states or outputs that can be reused once a target is known. SCB resists such attacks because its

cost functions depend directly on the seed, personalization string, cpucost, and memcost. Even small differences in these values cause the memory buffer to be filled completely differently and produce distinct sequences of derived keys.

As a result, assembling tables of precomputed SCB outputs is expensive and unlikely to be profitable. For password-based deployments, the addition of a salt or personalization value makes dictionary construction even less effective.

## **10.8 Side Channel Considerations**

SCB does not include dedicated side channel hardening beyond constant structure and deterministic sequencing. However, several design characteristics provide incidental resistance:

- memory operations are determined solely by memcost, not by secret data
- timing variation is dominated by high latency memory accesses, which reduces fine grained timing leakage
- sensitive values are cleared when no longer needed

Although SCB's design naturally resists some forms of timing and microarchitectural attacks, it should not be considered side channel hardened. Deployments requiring protection against local physical adversaries should use additional countermeasures.

## **10.9 Summary of Security Properties**

SCB provides strong, tunable resistance against brute force search, parallel hardware evaluation, and time memory tradeoff attacks. Its serial dependency chain prevents speculative evaluation and ensures that attackers must reproduce every stage of the derivation process. Memory hardness and computational hardness reinforce each other, creating a combined barrier that is difficult to bypass.

The implementation maintains all required security properties and does not introduce weaknesses through missing steps, incorrect sequencing, or unsafe memory handling. When configured correctly, SCB functions as a robust and practical cost based key derivation function suitable for high assurance environments.

# **11. Performance Characterization**

## 11.1 Overview

The performance profile of SCB is shaped by the interaction of two main cost levers: the memory required for each iteration and the number of iterations specified by `cpucost`. These two factors determine the total amount of memory traffic, the number of SHAKE based permutations, and the number of passes over the working buffer. SCB's design intentionally emphasizes memory latency and memory bandwidth as primary bottlenecks, ensuring that attackers cannot rely on fast arithmetic units to compensate for insufficient memory performance.

## 11.2 Influence of Memory Cost on Performance

The `memcost` parameter has the most significant impact on SCB runtime. Because the working buffer is sized in mebibytes and divided into fixed size cache line segments, increasing `memcost` increases both the number of segments that must be generated and the amount of memory that must be repeatedly touched during each iteration.

A larger buffer increases the number of scattered accesses required to complete the memory fill process. Each access incurs a high latency memory operation because the scattering pattern defeats CPU caching strategies. As `memcost` grows, these accesses dominate total runtime. Even systems with high memory bandwidth experience slowdowns, because the design is bound by latency rather than throughput.

The full buffer absorption step also grows with `memcost`. The implementation reprocesses the entire working buffer at regular intervals, producing substantial additional memory traffic during each iteration. As buffer size increases, these operations become proportionally more expensive.

## 11.3 Influence of Computational Cost

The `cpucost` parameter determines how many times the full memory fill and absorption process must be repeated. Its effect on total runtime is linear. Doubling `cpucost` doubles the number of working SHAKE initializations, memory fill cycles, index absorption operations, and derived key updates.

The computational cost consists primarily of Keccak permutations required for SHAKE output generation and for absorbing memory indices and buffer contents. Although these operations are relatively fast compared to memory accesses, they occur frequently enough to produce noticeable overhead at higher `cpucost` values. The impact becomes

especially clear when memcost is small, because the algorithm spends more time performing permutations and less time waiting on memory.

#### **11.4 Cache and Memory System Behavior**

SCB is intentionally designed to frustrate caching logic. The scattering pattern forces writes to memory locations separated by a distance intended to exceed the L2 cache capacity. As a result, almost every memory access bypasses both L1 and L2 caches. Prefetching strategies offer little benefit because access patterns are nonlocal and do not form predictable sequences.

This behavior makes SCB sensitive to the performance characteristics of main memory rather than the performance of the processor core. Systems with faster memory controllers perform better, and systems with limited bandwidth or shared memory experience noticeable slowdowns. This performance coupling is a deliberate part of the design. It ensures that attackers cannot rely on powerful arithmetic hardware to mask memory constraints.

#### **11.5 Performance Across Architectures**

SCB exhibits different performance patterns across different classes of hardware. On modern desktop processors with large caches and high memory bandwidth, SCB executes smoothly but still experiences the intended cache miss pattern. On mobile or embedded processors, which often have smaller caches and slower memory, SCB's performance characteristics become more pronounced, with a larger percentage of total runtime spent waiting on memory.

GPUs and dedicated accelerators are affected even more strongly. Their memory architectures are optimized for coalesced access patterns and predictable locality. SCB eliminates locality entirely, causing these platforms to perform poorly when evaluating scattered memory accesses. This makes high level parallelization strategies ineffective in practice.

#### **11.6 Impact of SHAKE Variant on Performance**

The seed length determines whether SCB uses the SHAKE 256 or SHAKE 512 rate. SHAKE 256 offers a higher rate of output per permutation, while SHAKE 512 requires more permutations to produce the same amount of data. As a result, SCB configurations using 64-byte seeds incur slightly higher computational cost during memory filling.

For memory heavy configurations, the difference is small relative to memory latency. For memory light configurations, the difference becomes more visible as the proportion of time spent performing sponge permutations increases.

### **11.7 Deterministic Performance Characteristics**

SCB performance is deterministic. The number of memory accesses, the number of permutations, and the volume of buffer absorption operations are fixed for a given combination of cpucost and memcost. There are no data dependent branches or variable length loops aside from output expansion. This predictability allows implementers to estimate runtime accurately for different parameter combinations without unexpected variation.

### **11.8 Performance Tuning Considerations**

Applications can tune performance by selecting appropriate values for cpucost and memcost. The optimal values depend on the target system's capabilities and the threat model of the deployment. Systems that prioritize responsiveness may choose smaller memcost values and moderate cpucost values, while systems that process keys infrequently can afford significantly larger values.

The deterministic and largely linear scaling behavior makes SCB predictable to deploy and straightforward to integrate into performance sensitive applications.

### **11.9 Summary**

SCB demonstrates performance characteristics that reflect its design goals. It is memory bound in most configurations, computationally linear in cpucost, and highly resistant to parallel acceleration. The scattered memory access pattern ensures that systems with high arithmetic capability do not gain disproportionate advantage, and the sequential structure ensures that attackers cannot pipeline or reorder operations to reduce cost. These properties support SCB's role as a robust cost based key derivation function.

## **12. Implementation Risks and Operational Limitations**

### **12.1 Overview**

Although SCB is a robust and internally consistent construction, its effectiveness depends on correct parameter selection, proper resource availability, and awareness of

environmental constraints. The implementation faithfully enforces the design, but like any cost based key derivation function, SCB introduces operational risks when deployed without a clear understanding of system limits and application requirements. This chapter examines those risks and discusses conditions under which SCB must be configured carefully in order to remain secure and practical.

## **12.2 Parameter Misconfiguration Risks**

The most common risk in deploying SCB arises from selecting inappropriate values for `cpucost` or `memcost`. These parameters define the hardness profile of the function. If either value is set too low, SCB may execute quickly but fail to provide sufficient resistance against brute force or parallelized attacks. If either value is set too high, SCB may consume excessive time or memory, causing service degradation or application timeouts.

Because SCB does not attempt to select parameter values automatically, it is the responsibility of the caller to understand the system's memory availability and the desired security level. Inconsistent or overly aggressive parameter choices can create significant operational friction.

## **12.3 Memory Allocation Failures and Resource Constraints**

SCB must allocate a buffer equal to `memcost` mebibytes during each derivation. On systems with limited memory, this allocation may fail or cause the system to resort to paging. In such cases, performance can degrade sharply, and the added system pressure may impact unrelated processes.

The implementation handles allocation failures safely, but persistent memory pressure can create unpredictable behavior at the application level. Developers integrating SCB should validate that the chosen `memcost` values are compatible with the system's available physical memory and its operational load patterns.

## **12.4 Runtime Considerations and System Responsiveness**

Because SCB enforces a fixed amount of computation and memory traffic per iteration, it can create latency concerns in real time or user responsive environments. The scattered memory operations consume large amounts of time relative to purely computational functions. This behavior is desirable for security but must be balanced against user experience requirements.

In multi-tenant environments, large SCB operations may create contention in shared memory subsystems. Although this does not affect correctness, it may reduce throughput or introduce timing variability in other applications on the same system.

## **12.5 Limitations in Side Channel Behavior**

SCB is not designed as a side channel hardened primitive. While it does benefit from the nondeterministic nature of memory latency patterns, its primary goal is to enforce cost and serial dependency rather than to obscure timing behavior. This means that systems with strict side channel resistance requirements may need additional countermeasures.

The memory scattering pattern is independent of secret material, which avoids exposure of secret dependent indexing. However, the volume of memory traffic and the iterative structure may still produce observable timing signals that advanced attackers could exploit in tightly controlled environments.

## **12.6 Variation Across Hardware Architectures**

Different hardware platforms exhibit different memory and cache characteristics. Some processors have larger L2 or L3 caches, while others have shallow memory hierarchies or unified memory designs. Although SCB enforces scattering patterns that exceed typical cache sizes, performance and timing behavior still vary across platforms.

On embedded devices, where memory bandwidth and clock frequency are limited, SCB may consume considerable runtime even at moderate memcost values. The implementation remains correct in these environments, but practical deployment requires careful parameter selection to ensure a reasonable balance between latency and security.

## **12.7 Operational Costs in High Throughput Environments**

SCB is effective as a cost based key derivation function, but its design inherently limits throughput. High volume services that derive many keys per second must consider the impact of repeated large buffer allocations and memory fill cycles. The serial nature of SCB prevents pipelining or batch optimization. Each derivation must proceed from start to finish before the next can begin.

For high throughput systems, implementers may choose to restrict SCB to operations that occur infrequently, such as password verification or long-term key derivation, rather than integrating it into frequent handshake or session level operations.

## **12.8 Risks Associated With Update Semantics**

SCB includes an update interface that allows callers to inject new seed material into an existing context. While structurally valid, this feature must be used with care. Repeated updates without well-defined semantics may produce sequences of derived keys that are difficult to track or validate. Updates also do not reset cost parameters, which could cause inconsistencies if operational requirements change over time.

Implementers should clearly define when updates are permitted and how updated state interacts with application logic, particularly in systems where the derived key influences additional cryptographic operations.

## **12.9 Responsibility for Seed and Personalization Management**

SCB assumes that the caller manages seed material securely. Weak or low entropy seeds reduce SCB's security profile regardless of cost parameters. Similarly, incorrect or inconsistent use of personalization strings may undermine domain separation or produce unintended collisions across application contexts.

The implementation cannot compensate for these issues. Applications must ensure that seeds are generated securely and that personalization values reflect the correct operational scope.

## **12.10 Summary of Limitations and Risks**

The SCB implementation is correct and secure within its defined operational model. However, its real-world effectiveness depends on careful selection of cost parameters, safe handling of seed material, and awareness of system memory constraints. SCB is most effective in environments where high latency key derivation is acceptable and where memory resources are stable and predictable.

When deployed responsibly, SCB provides strong security benefits. When deployed without attention to operational constraints, it may introduce unintended performance and reliability issues. These risks can be managed through informed parameter selection and system level planning.

# **13. Deployment Recommendations**

## **13.1 Overview**



SCB is designed to impose meaningful cost on adversaries who attempt to recover secrets through brute force or large-scale parallel search. Its effectiveness depends not only on its internal correctness but also on how it is deployed in real systems. This chapter provides guidance for selecting cost parameters, managing seeds and personalization strings, and integrating SCB into broader security architectures. The recommendations presented here reflect practical considerations arising from SCB's memory and computational characteristics.

## **13.2 Selecting Memory and CPU Cost Parameters**

The choice of memcost and cpucost values determines SCB's hardness profile and must be informed by the hardware characteristics of the system in which it is deployed.

### **Memory Cost Selection**

memcost directly controls the size of the working buffer. Reasonable values depend on system capabilities:

- Desktop and server systems typically support larger memcost settings and may use values between 64 and 256 when strong resistance to brute force is required.
- Embedded or resource constrained devices may require smaller values to maintain responsiveness. Even so, memcost should not be reduced below levels that compromise security goals.
- Multi-tenant systems must consider aggregate memory usage when many SCB operations run concurrently.

### **CPU Cost Selection**

cpucost determines how many iterations SCB must complete. Higher values increase serial work and improve resistance to parallel attack strategies.

- For interactive applications, cpucost values in the lower range provide reasonable latency.
- For batch or high assurance systems, larger values increase attacker cost without significantly affecting throughput.

The relationship between cpucost and total runtime is linear, making it straightforward to tune performance empirically.

### 13.3 Seed Handling and Entropy Requirements

SCB assumes that callers provide seed material that meets the entropy requirements of the application. For password-based deployments, seed material should be produced through a controlled preprocessing step that includes salting, normalization, and conversion of user input into a stable internal representation.

Seeds of 32 or 64 bytes must be constructed with care. Weak or predictable seeds diminish the security that SCB provides, regardless of cost parameter selection. It is essential that all systems using SCB incorporate a mechanism for generating high quality randomness or for strengthening low entropy secrets before passing them to SCB.

### 13.4 Personalization and Domain Separation

The optional personalization string offers a straightforward method for ensuring domain separation. Systems that use SCB for unrelated purposes should choose distinct personalization values to avoid accidental key overlap.

Examples include separating:

- user authentication workflows
- encrypted storage subsystems
- internal cryptographic services
- application specific protocol layers

Even when seed values are unrelated, the use of personalization strings contributes to a safer and more organized cryptographic architecture.

### 13.5 Deployment in Password Based Systems

SCB is particularly well suited to password processing due to its cost-based nature. When used in this context, deployments should establish policies that record the memcost and cpucost values used for each password derivation so that verification can be performed consistently.

Salts or appropriate personalization values should also be stored and managed securely. Password based deployments should ensure that increases in memcost or cpucost can

be applied gradually as hardware capabilities improve, without invalidating existing stored credentials.

### **13.6 Integration Into Layered Key Derivation Architectures**

In multi stage key derivation pipelines, SCB can serve either as a first layer that strengthens low entropy inputs or as a standalone KDF. When placed before other derivation stages, SCB provides an expensive transformation that improves the overall security of the system. Subsequent stages may use HKDF or domain separated SHAKE to produce application specific keys.

Because SCB is deterministic, integration with other deterministic KDFs is straightforward and does not introduce ambiguity into the resulting key hierarchy.

### **13.7 Operational Guidance for High Security Systems**

Systems requiring long term confidentiality or high resistance to hardware accelerated attacks may benefit from higher memcost values. These deployments should perform careful benchmarking to ensure that runtime remains acceptable, particularly for operations that occur frequently.

It is useful to document the chosen parameters and associated hardware assumptions as part of the system's security profile. This helps ensure that SCB continues to meet security objectives as hardware platforms evolve.

### **13.8 Managing Updates and Context Lifecycle**

SCB includes functionality that allows callers to inject additional seed material into an existing context. This feature is useful for refreshing state or mixing new entropy into ongoing processes, but it should be used deliberately. Update operations do not reset cost parameters, and repeated updates without clear purpose may complicate analysis or produce confusion regarding derived key provenance.

Developers should treat SCB contexts as single use objects unless clear update semantics are defined. After completion, contexts should be disposed to prevent unintended reuse.

### **13.9 Avoiding Common Pitfalls**

Several deployment mistakes can reduce SCB's effectiveness:

- Choosing memcost or cpucost values that are too small for the threat model.
- Using seeds with insufficient entropy.
- Inconsistent or incorrect use of personalization strings.
- Deploying SCB in high throughput systems without considering memory load.
- Allowing contexts to persist or be reused without proper disposal.

Avoiding these pitfalls ensures that SCB operates as intended and that its memory and computational hardness properties are preserved.

### **13.10 Summary**

Deploying SCB effectively requires an understanding of its cost profile and an awareness of system resource constraints. With appropriate parameter selection, secure seed handling, and consistent operational practices, SCB offers a robust and reliable method for deriving hardened cryptographic keys. When misused or under configured, SCB may still operate correctly but fail to deliver the security benefits expected from a cost-based design.

## **14. Conclusion**

### **14.1 Summary of Findings**

This analysis has examined the SCB construction from both a structural and an implementation perspective, drawing on the specification and the reference code to evaluate correctness, consistency, and practical security properties. SCB demonstrates a coherent integration of memory hardness, computational hardness, and deterministic state evolution. The algorithm's design ensures that each derivation incurs a substantial and irreducible amount of memory traffic and permutation driven computation. The implementation enforces these properties with accuracy, precision, and predictable behavior.

Across all examined components, SCB adheres to its specification. The memory filling engine, the scattering pattern, the iterative hash state, and the final expansion phase all reflect the intended architecture. No deviations or shortcuts appear in the reference

implementation, and no inconsistencies arise from parameter handling, state transitions, or memory management.

## **14.2 Evaluation of Security Objectives**

SCB successfully achieves its goal of resisting brute force search, parallelized hardware attacks, and time memory tradeoff strategies. Its memory hardness ensures that attackers must incur the cost of a large working buffer, while its computational hardness guarantees that a full sequence of Keccak permutations must be carried out in a strict and non-parallelizable order. The sequential structure prevents speculative evaluation and eliminates opportunities for pipelined processing. Together, these properties support the use of SCB in applications where resistance to large scale adversaries is essential.

The deterministic nature of the construction ensures that SCB can be reproduced reliably across platforms, making it suitable for systems that require consistent and verifiable key derivation.

## **14.3 Considerations for Practical Deployment**

While SCB meets its security objectives, it must be configured responsibly. The choice of memcost and cpucost values determines the function's real world cost profile. High values produce strong resistance at the cost of additional runtime and memory consumption, while low values may undermine the intended hardness. For this reason, deployments should select parameters that reflect both system capabilities and application-level threat models.

Memory availability, expected throughput, and user responsiveness are important considerations. SCB is most effective in environments where key derivation is infrequent or where high-cost operations are acceptable.

## **14.4 Final Assessment**

The SCB design and implementation form a well-structured and reliable cost based key derivation function. The construction adheres to modern best practices for memory hard KDFs and leverages the well understood Keccak sponge at its core. The reference implementation reflects the design with high fidelity, enforces strict sequencing of operations, and demonstrates disciplined handling of memory and internal state.

When deployed using appropriate parameter values and secure seed handling practices, SCB provides robust protection against brute force and hardware accelerated attacks. It stands as a clear and consistent implementation of a memory hard KDF built on SHAKE, and it satisfies the requirements expected of a hardened key derivation mechanism.

## Chapter 20: References and Links

This section lists the normative and informative references used in the development, verification, and analysis of the SHAKE Cost-Based Key Derivation Function (SCB). All sources are publicly available and verifiable through stable institutional repositories. Where applicable, direct URLs to authoritative copies are provided to ensure reproducibility and citation integrity.

### 20.1 Standards and Specifications

1. NIST FIPS 202 – SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions  
National Institute of Standards and Technology (NIST), August 2015.  
<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf>
2. NIST SP 800-185 – SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, and ParallelHash  
National Institute of Standards and Technology (NIST), December 2016.  
<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-185.pdf>
3. NIST SP 800-132 – Recommendation for Password-Based Key Derivation  
National Institute of Standards and Technology (NIST), December 2010.  
<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-132.pdf>
4. ISO/IEC 19790:2012 – Security Requirements for Cryptographic Modules  
International Organization for Standardization, 2012.
5. ISO/IEC 27001:2022 – Information Security, Cybersecurity, and Privacy Protection – Information Security Management Systems  
International Organization for Standardization, 2022.

6. Percival, C. *Stronger Key Derivation via Sequential Memory-Hard Functions (scrypt)*. Self-published technical report, 2009.  
<https://www.tarsnap.com/scrypt/scrypt.pdf>
7. Biryukov, A., Dinu, D., Khovratovich, D. *Argon2: The Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. University of Luxembourg, 2016.  
<https://www.ietf.org/archive/id/draft-irtf-cfrg-argon2-10.txt>
8. Boneh, D., Corrigan-Gibbs, H. *Balloon Hashing: A Memory-Hard Function Providing Provable Protection Against Sequential Attacks*. Proceedings of ASIACRYPT 2016, Springer LNCS 10031.
9. Forler, C., Lucks, S., Wenzel, J. *Catena: A Memory-Consuming Password Scrambler*. Password Hashing Competition submission, 2014.  
<https://password-hashing.net/submissions/specs/Catena-v3.pdf>
10. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G. *Keccak Specifications*. Submission to NIST SHA-3 competition, version 3, January 2011.  
[https://keccak.team/keccak\\_specs\\_summary.html](https://keccak.team/keccak_specs_summary.html)
11. European Union. *General Data Protection Regulation (GDPR), Regulation (EU) 2016/679*.  
Official Journal of the European Union, May 2016.  
<https://eur-lex.europa.eu/eli/reg/2016/679/oj>
12. U.S. Department of Health and Human Services. *Health Insurance Portability and Accountability Act (HIPAA) Security Rule, 45 CFR Part 164*.  
<https://www.hhs.gov/hipaa/for-professionals/security/index.html>
13. U.S. Congress. *Federal Information Security Modernization Act (FISMA) of 2014*.  
Public Law 113-283.  
<https://www.congress.gov/bill/113th-congress/house-bill/1163>
14. International Organization for Standardization. *ISO/IEC 27701:2019 – Privacy Information Management*.  
Geneva, Switzerland, 2019.
15. QRCS Corporation. *QSC Cryptographic Library - SCB Implementation Files*.  
GitHub Repository: <https://github.com/QRCS-CORP/QSC>